



Building Alerts for AI System Failures

From Reactive Firefighting to Proactive Reliability

A practical guide for SREs, ML engineers, and engineering managers responsible for maintaining reliable AI systems.

Why Alerts Matter

AI systems fail in **subtle & silent ways**, often without the obvious crashes we see in traditional systems.

Early detection prevents:

Customer-Facing Outages

Catch problems before users experience them

Model Drift Going Unnoticed

Prevent gradual degradation of performance

Expensive Infrastructure Waste

Avoid unnecessary cloud compute costs



Alerts = the bridge between monitoring and action

Types of Failures to Alert On

1

Infrastructure

- GPU out-of-memory errors
- Thermal throttling events
- Node crashes or restarts
- Network connectivity issues

2

Operational

- High p95/p99 latency spikes
- Error rate increases
- Request throughput anomalies
- Batch job failures

3

Model-level

- Accuracy/precision drops
- Feature drift signals
- Bias or fairness issues
- Prediction confidence changes

4

Business-level

- KPI degradation (CTR)
- Fraud rate increases
- Customer churn signals
- Revenue impact indicators

Anatomy of a Good Alert

Signal

The specific metric, log pattern, or event that triggers the alert

Example: `model_inference_latency_p95 > threshold`

Threshold

Static: Fixed value (>80% GPU for 10m)

Dynamic: SLO-based (2x baseline, historical patterns)

Actionability

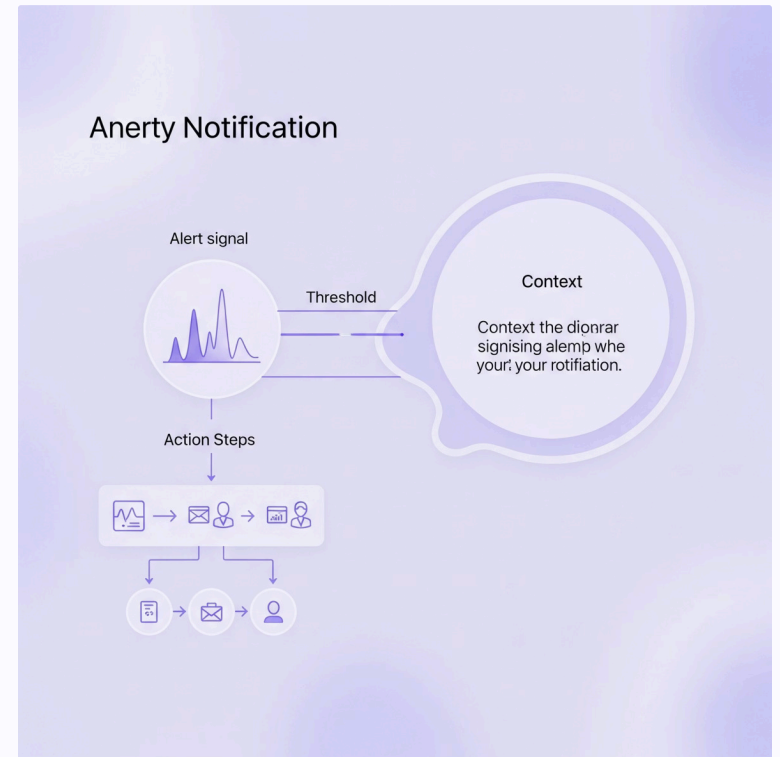
Clear next steps for on-call engineers

Link to runbook or troubleshooting guide

Context

Include model, version, cluster, team labels

Relevant metrics and recent changes



"An alert that doesn't lead to action is just noise."



Prometheus Alerting Example

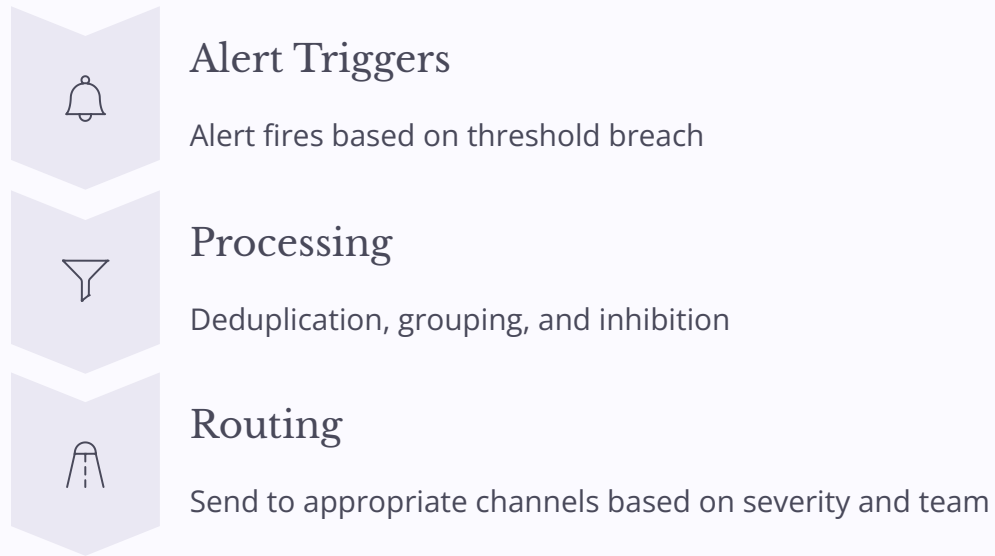
A practical example of an alert definition in Prometheus alert.rules.yml:

```
groups:
- name: ai-infra
  rules:
  - alert: HighGPULatency
    expr: histogram_quantile(0.95,
      rate(inference_latency_bucket[5m])) > 200
    for: 5m
    labels:
      severity: critical
      team: ml-platform
      component: inference
    annotations:
      summary: "High GPU inference latency"
      description: "p95 >200ms on {{ $labels.model }}
        in {{ $labels.cluster }}"
      runbook: "https://docs.example.com/runbooks/high-latency"
```

Pro Tip: Use templating in alert descriptions to include the specific instance, service, and threshold that triggered the alert.

Routing Alerts

Using Alertmanager for deduplication & intelligent routing



Use **severity levels** to determine urgency:

- **Critical** - Immediate action (PagerDuty)
- **Warning** - Attention needed (Slack)
- **Info** - Awareness only (Email)



Avoiding Alert Fatigue

Focus on SLOs, Not Every Metric

Alert on service level objectives that matter to users, not on every possible metric fluctuation

Use Multi-Window Thresholds

Combine short-term sensitivity (5m) with longer-term confirmation (1h) to reduce false positives

Group Related Alerts

Consolidate multiple related issues into a single notification to avoid alert storms during incidents

Regular Alert Hygiene

Conduct monthly reviews of alert patterns, tuning or removing alerts that are consistently noisy

⊗ Teams receiving more than 5-8 actionable alerts per day will start to ignore them.



AI-Specific Alerting Patterns



Model Drift Detection

Alert when feature distributions or prediction patterns deviate significantly from baseline



Data Pipeline Stalls

Detect when no new training data has been processed within expected timeframe



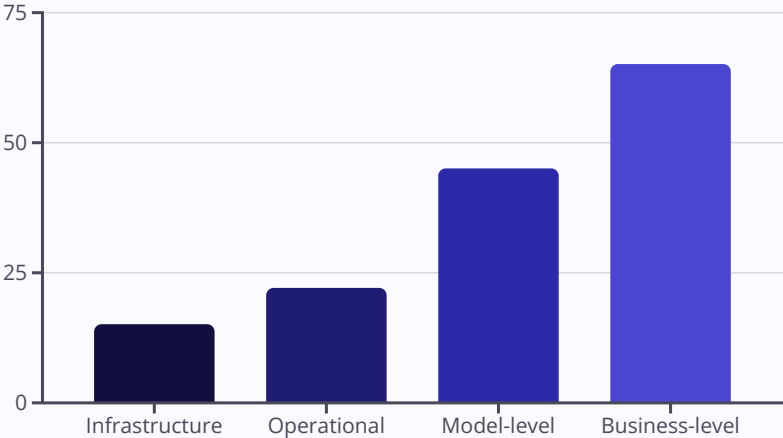
GPU Underutilization

Identify expensive idle resources that could be reallocated or scaled down



Shadow Model Disagreement

Alert when canary/shadow models diverge significantly from production model



Higher-level alerts typically require more investigation time but prevent costlier failures.

Incident Playbooks

Each alert should link to a [detailed runbook](#) with step-by-step troubleshooting instructions.

Example: High Inference Latency Playbook

01

Check HPA Scaling Status

Verify if autoscaler is functioning and if pods are scaling as expected

02

Inspect GPU Utilization with DCGM

Check for thermal throttling, memory pressure, or bottlenecks

03

Analyze Request Patterns

Look for unusual input sizes, batch sizes, or request patterns

04

Roll Back Latest Model if Needed

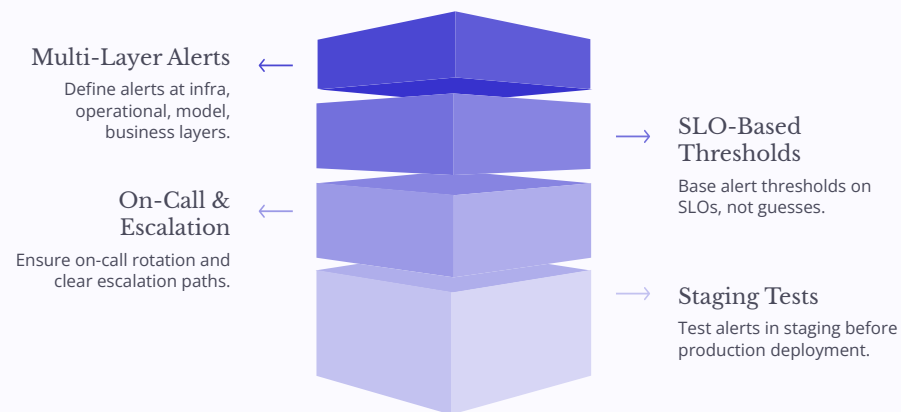
Revert to previous stable version if issue persists



Well-documented playbooks reduce Mean Time To Resolution (MTTR) by up to 70%.

✓ **Tip:** Update playbooks after each incident with new learnings.

Best Practices



Define Multi-Layer Coverage

Ensure alerts exist at all levels: infrastructure, operational, model performance, and business impact.

Establish Clear On-Call Practices

Document who responds to what, when, and how escalation works. Include follow-the-sun rotation for global teams.

Use SLO-Based Thresholds

Set thresholds based on customer experience goals, not arbitrary values or technical guesses.

Test Before Production

Validate alerts in staging environments to prevent false positives and ensure proper routing.



Key Takeaways

Turn Monitoring into Action

Alerts transform passive monitoring data into actionable signals that drive response

Cover All Failure Modes

Implement alerts across infrastructure, operational, model, and business layers

Fight Alert Fatigue

Prioritize SLO-based, actionable alerts and regularly prune noisy ones

Enable Rapid Recovery

Combine alerts with detailed runbooks to decrease MTTR and improve reliability

Questions? Contact reliability-team@example.com